University of Windsor

Faculty of Engineering

Department of Electrical and Computer Engineering

Bus Transit Simulation (Topic #6) – Design Document

Team #20

Misk Damdoum (110164166) Gurjashan Sidhu (110173740)

Malik Ramadan (110167718) Emma Khuong (110170759)

Rayan Charbaji (110176892) Kais Aldehneh (110177627)

Software Fundamentals (ELEC 2320)

Instructor: Ayobami Adewale

March 2, 2026

1. Introduction

1.1 Problem Statement

A city transit authority is facing challenges with overcrowding, long wait times, and service delays on a busy bus route. To understand these issues, this project develops an object oriented simulation of buses operating on an 18-stop loop. The simulation is used to analyze how scheduling and operational factors affect system performance by analyzing outcomes such as passenger wait time, trip duration, bus load, passengers left behind, and schedule reliability. This allows city improvements to be reviewed without making immediate real world changes.

1.2 Project Scope

This project focuses on simulating how a bus transit system operates along an 18-stop loop. It includes modeling passenger arrivals, route setup, and how buses move between stops, as well as collecting performance data to understand how well the system runs.

The simulation specifically includes:

- Passenger arrivals at stops (time, origin, destination, payment)
- Route configuration (number of buses, headways, travel times)
- Bus movement between all stops (2-5min)
- Boarding and alighting processes (card/tap vs cash)
- Capacity limits for buses
- Performance metrics such as: Passenger wait time, total trip time, bus load, passengers left behind and schedule delays

To keep the project focused, the simulation does not include features such as GPS tracking, graphical user interfaces, or real-time database integration, as these are beyond the purpose of evaluating the systems performance.

2. System Design

2.1 Main Classes

<u>Class: Passenger</u>

Responsibilities:

The Passenger class represents a single rider in the system. It stores all information related to that rider's trip and tracks their progress from arrival at the stop to arrival at their destination.

Each passenger:

- Arrives at a specific stop at a specific time
- Waits in that stop's queue
- Boards a bus if space is available
- Alights at their destination stop

This class also stores timestamps used to compute average wait time and total trip time.

Relationships:

- A Passenger waits at one Stop
- A Passenger boards one Bus
- A Passenger is carried by a Bus until reaching their destination Stop

<u>Class: Stop</u>

Responsibilities:

The Stop class represents one of the 18 stops in the loop. Its main job is to manage the queue of waiting passengers and interact with buses during boarding and alighting.

The Stop must:

- Maintain a queue of passengers waiting at the stop, ensuring they are boarded in the order they arrived
- Allow boarding when a bus arrives (subject to capacity)
- Track how many passengers are left behind if a bus is full
- Ensure that only one bus occupies the stop at a time

If a second bus arrives while the stop is occupied, it must wait until the first bus clears.

Relationships:

- Contains multiple Passenger objects in its queue.
- Interacts with Bus objects during arrival and departure.
- Managed by the Simulation class.

<u>Class: Bus</u>

Responsibilities:

The Bus class represents a vehicle operating on the route. It tracks its position, onboard passengers, capacity, and schedule adherence.

When a bus arrives at a stop:

1. Passengers whose destination matches the stop alight (2 seconds each).
2. Waiting passengers board if capacity allows:
    - 4 seconds per tap passenger
    - 6 seconds per cash passenger

3. The total dwell time is calculated based on these boarding and alighting times.
4. The bus does not depart until all boarding and alighting is complete.

The bus also compares its actual arrival time with its scheduled arrival time. If it falls more than 3 minutes (180 seconds) behind schedule, it may skip stops where:

- No passenger needs to alight.
- No passenger is waiting to board.

Relationships:

- Carries multiple Passenger objects.
- Visits Stop objects in sequence.
- Uses Route data to determine travel times.
- Controlled by Simulation.

<u>Class: Route</u>

Responsibilities:

The Route class stores the configuration of the route for a simulation run. It contains all static information needed for buses to operate correctly.

This includes:

- Number of buses in service
- Headway between departures from stop 1
- Travel time between consecutive stops
- Bus capacity

It also supports the 18-stop loop by providing logic for moving from stop 18 back to stop 1.

Relationships:

- Used by Bus to determine movement and schedule.
- Owned by Simulation.

<u>Class: Simulation</u>

The Simulation class is the core controller of the system. It:

- Reads the passenger file and route configuration file
- Creates Stop, Bus, and Passenger objects
- Manages the simulation clock
- Processes events in chronological order
- Enforces all system rules:
    - Boarding/alighting timing
    - Capacity constraints
    - Skip-stop logic

    - Stop occupancy rules
- Collects performance metrics

Metrics Collected:

1. Average passenger wait time
2. Average total trip time
3. Bus load factor at each stop
4. Total passengers left behind
5. Lateness of each bus over time

2.2 UML Diagram

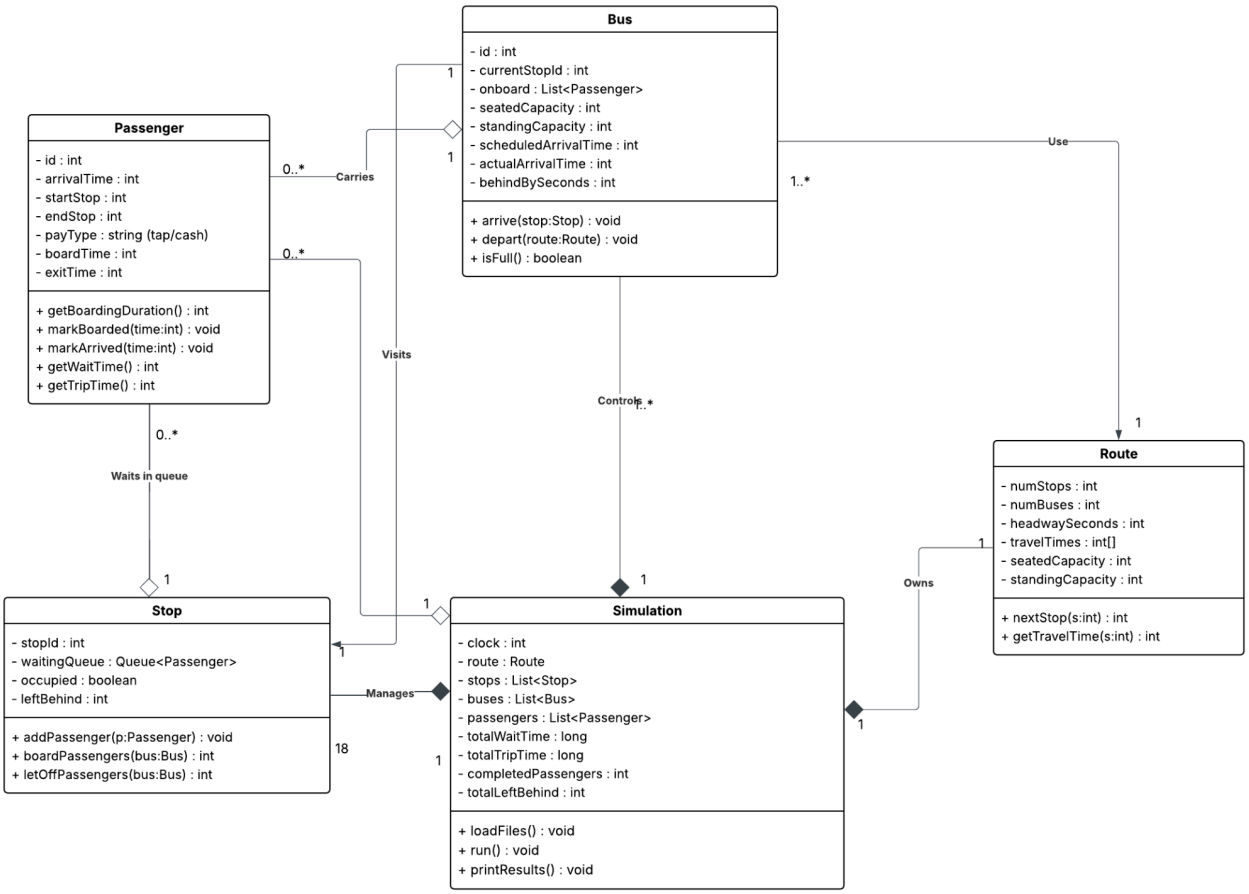

Figure 1: UML Class Diagram of the Transit Bus Simulation System

This UML diagram shows the overall structure of the bus simulation system and how its main components interact. The system is built around five core classes: Simulation, Route, Stop, Bus, and Passenger, each with a clear responsibility.

At the center is the Simulation class, which acts as the main controller. It manages the simulation clock, creates the route, stops, buses, and passengers, and processes events like bus arrivals and passenger boarding. It also tracks performance metrics such as total wait time, trip time, and passengers left behind.

Because it creates and manages Route, Stop, and Bus objects, these relationships are shown using composition (filled black diamonds).

The Route class defines the system setup, including the number of stops, number of buses, departure headway, travel times between stops, and bus capacity. These values remain constant during the simulation, acting as a fixed blueprint that buses follow.

The Stop class represents one of the 18 stops in the loop. Each stop maintains a queue of waiting passengers and ensures only one bus occupies it at a time. When a bus arrives, the stop manages boarding and alighting and records any passengers left behind. Since passengers are only temporarily at a stop, the Stop–Passenger relationship is aggregation (hollow diamond).

The Bus class represents a vehicle on the route. It tracks its current stop, onboard passengers, capacity, and timing for schedule adherence. At each stop, it handles alighting, boarding, dwell time, and possible stop-skipping if behind schedule. Its relationship with Passenger is also aggregation, as passengers are only temporarily carried.

Finally, the Passenger class represents individual riders. Each passenger has an arrival time, origin and destination stops, payment type, and timestamps used to calculate wait and trip time. A passenger starts at a stop, boards a bus if space is available, and exits at their destination.

2.3 Flowchart

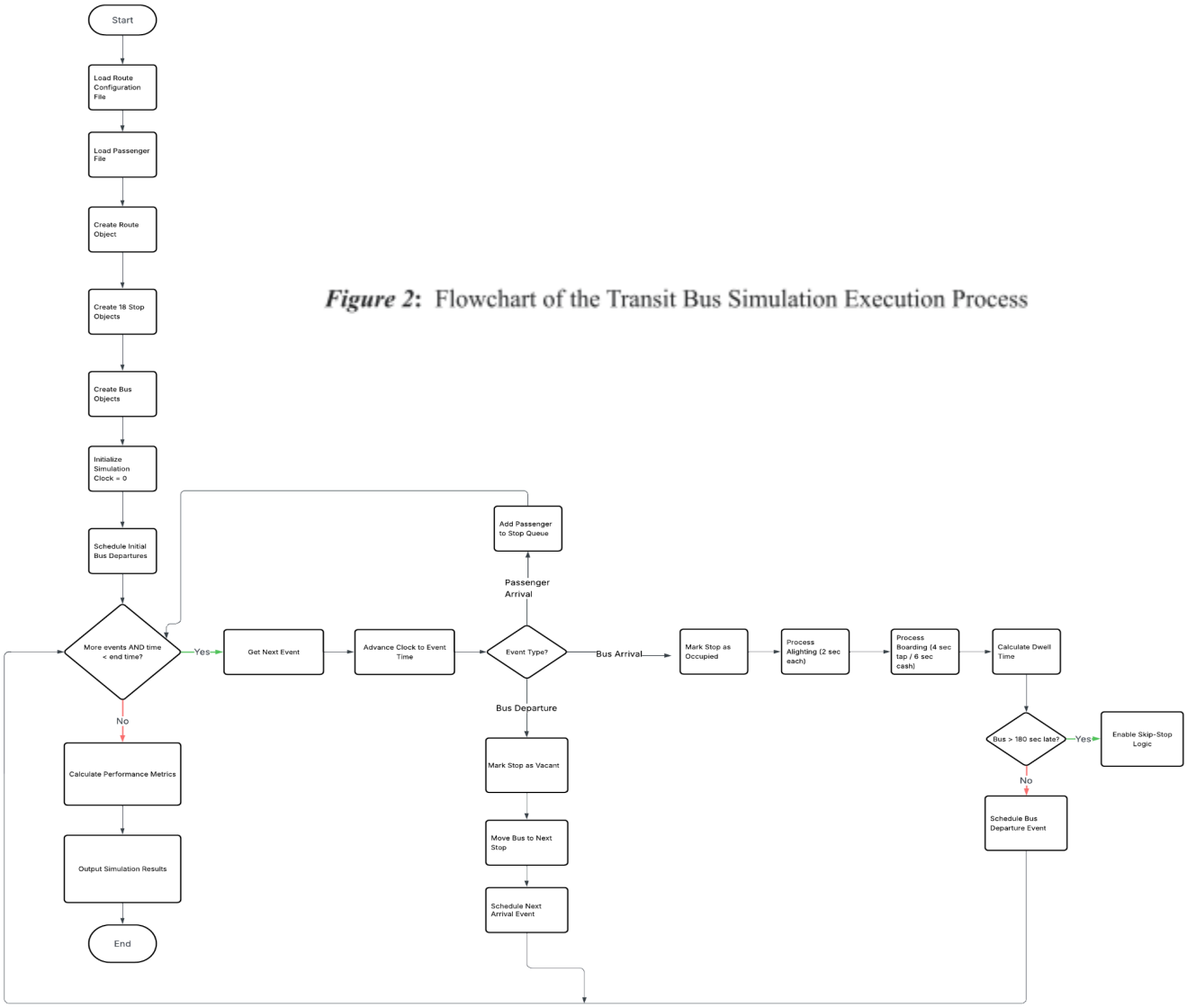

***Figure 2*:** Flowchart of the Transit Bus Simulation Execution Process

3. Requirements

3.1 Functional requirements:

1. Generate passenger data including arrival time, origin stop, destination stop, and payment method.
2. Generate route data including number of buses, departure intervals, travel times between stops, and bus capacity.
3. Simulate bus operations across all stops, including:
    - Passenger boarding and alighting times
    - Bus capacity limits
    - Waiting if early and detecting delays

    - Skipping stops when no passengers are boarding or alighting
    - Preventing buses from passing each other
4. Calculate and report:
    - Average passenger wait time
    - Average trip time
    - Bus load factor at each stop
    - Number of passengers left behind
    - How far each bus falls behind schedule

3.2 Non functional requirements:

The system must:

1. Accurately track time and passenger movements.
2. Handle multiple buses and passengers efficiently.
3. Separate passenger generator, route generator, and simulation program.
4. Allow different route and passenger configurations for comparison.
5. Produce clear and easy-to-understand output results.

4. Algorithms

Objective: Develop an algorithm that generates a reproducible list of passengers for the simulation process based on arrival time, origin, destination, and type of payment.

4.1 Passenger Arrival Algorithm

<u>Steps</u>

1. Read the configuration file.
    - Total simulation time, passenger generation parameters, random seed
2. Initialize random number generator with random seed.
3. For each stop (1-18):
    - Generate passenger arrival times
    - For each passenger:
        - Set origin as the current stop
        - Select designation stop from the stops on the route (must not = origin and must be downstream from the current stop accordion to the loop direction of the route)
        - Select payment method (tap card with probability p; cash with probability (1 - p)
        - Ensure arrival times are non-decreasing

4. Write passenger data to output file
    - Arrival time, Origin, Destination, Method of payment
5. Close the file and end the generator.

<u>Assumptions</u>

- Passengers arrival is an independent process.
- The destination is selected uniformly.
- The probability of selecting a type of payment can be changed.
- If the same seed and input values are used, the same output will always be generated.

4.2 Bus Movement Algorithm

- Start the buses to the initial stop.
- Before the simulation duration limit is reached, move the bus to the next stop, and allow passengers to get off, then allowing passengers to board the bus until it is full or no passengers wait to board.
- If the bus is behind schedule, carry out the skip procedure and update the performance statistics.

4.3 Schedule Adherence Algorithm

- Compare the actual arrival with the scheduled arrival.
- Calculate the delay and cumulative delay.
- Determine if the bus is “behind schedule.”

5. Libraries and Tools

The bus transit simulation will use standard C++ libraries to ensure a modular and well-structured implementation. The **<iostream>** library will handle console input and output, while **<vector>** and **<queue>** will manage dynamic collections such as passenger lists and waiting lines to realistically simulate ticketing and boarding. The **<fstream>** library will support file input and output for storing passenger arrival data and route configuration details. The **<random>** library will generate pseudo-random numbers to determine passenger arrivals and trip types. Using a set random seed will allow simulation runs to be reproduced during testing and make it possible to compare results under different seeds. Additional libraries such as **<string>** and **<iomanip>** may be used for text handling and formatted statistical output. These libraries represent the primary tools identified during the design phase. As development progresses, additional libraries or structural refinements may be introduced to improve

performance. Version control will be managed alone using **GitHub** to support organized development and collaboration. It will allow the team to track individual contributions through commit history and manage code revisions. This way, the team can coordinate the integration of changes efficiently throughout the project.

6. Team Task Distribution

Team Member	Assigned Responsibilities
Misk Damdoum	Implement Passenger class and Passenger Arrival Algorithm (Section 4.1). Handle random seed logic and passenger file generation. Assist with testing reproducibility of results.
Gurjashan Sidhu	Implement Stop class including passenger queue management, boarding rules, left-behind tracking, and stop occupancy constraints.
Malik Ramadan	Implement Bus class including boarding/alighting timing logic, dwell time calculation, capacity handling, and skip-stop functionality.
Emma Khuong	Implement Route class and configuration file parsing. Handle headway setup, travel time storage, loop logic (Stop 18 → Stop 1), and bus scheduling initialization.
Rayan Charbaji	Develop Simulation controller class. Manage event processing, simulation clock, integration of all classes, and enforcement of system rules.
Kais Aldehneh	Implement performance metrics collection and reporting (average wait time, trip time, load factor, lateness tracking). Handle formatted output and statistical summaries.

Note: Tasks are subject to change as project progresses.

7. Conclusion

This project develops a simulation that models the operation of a bus transit system along an 18-stop loop. The system represents different elements such as passenger arrivals, bus movement, boarding and capacity limits. These are used to evaluate how operational factors will determine the overall performance of the system. The simulation shows different route and passenger arrangements, allowing for various scenarios to be tested. By adjusting inputs such as the number of buses, departure headways, and travel times the system's behaviour can be compared under different conditions without real world implementation. The structure of the system is built around separate classes for passengers, stops, buses, routes, and simulation control, which supports scalability. New configurations can be added without major changes to the overall design. In conclusion, the system achieves its goal of providing a structured tool for analyzing transit performance and supporting scheduling decisions.